

Templates in C++

1) What are templates?

- A **template** lets you write one function or class that works with **many data types**.
- Two kinds:
 - **Function templates** – generic functions.
 - **Class templates** – generic classes (containers like stack, list, arrays, etc.).
- Templates enable **generic programming**: one definition, many type-specific instantiations.

2) Why templates (vs. function overloading)?

- **Function overloading** = write the “same” operation for different types by duplicating functions.
- Drawbacks of overloading:
 1. Rewriting the same body for each type is time-consuming.
 2. Program uses more disk space (lots of near-duplicate code).
 3. Bug fixes must be repeated across all overloads.
- **Templates** solve this by letting you write **one generic definition** that the compiler reuses across types.

3) Basic template syntax

- Both typename and class keywords are valid and equivalent in parameter lists:

```
template <typename T>
ret_type function(args) { /* ... */ }

template <class T>
ret_type function(args) { /* ... */ }
```

4) How templates compile (instantiation)

- The compiler **can't generate code** from the template alone because type(s) are unknown.
- When it sees a **call/use**, it deduces types and **instantiates a concrete** version (replacing T with the actual type).
- It generates **one version per unique type** used in your program (no matter how many calls of that type).
- **Memory/code size**: Templates do **not inherently save memory**—separate instances are emitted per type (e.g., int, char, double). The benefit is **less source code** and **single-point maintenance**.

5) Function template: simple add

- Example:

```
template <class T>
void add(T a, T b) { std::cout << '\n' << (a + b) << '\n'; }
```

- Calls like add(20,40), add('A','b'), add(34.30,45.89) instantiate int, char, and double versions.
- Note: adding char uses **character codes** (prints numeric sum unless streamed as char).

6) Function templates with two generic types

- Use separate parameters if arguments may differ:

```
template <class T>
void add(T a, T b) { std::cout << '\n' << (a + b) << '\n'; }
```

- Don't reuse T1 for both parameters if the types can differ (using T1 a, T1 b would forbid int + double).

7) Requirements on operand types (operators)

- Template bodies compile **per instantiation**. For a + b to work, that pair of types must support operator+.
- For **user-defined types**, provide the needed operators (often both operator+ and operator<< for printing).

```
class MyClass {
    int num;
public:
    explicit MyClass(int k) : num(k) {}
    MyClass operator+(int k) const { return MyClass(num + k); }
    MyClass operator+(const MyClass& r) const { return MyClass(num + r.num); }
    friend MyClass operator+(int k, const MyClass& r) { return MyClass(k + r.num); }
    friend std::ostream& operator<<(std::ostream& o, const MyClass& r) { return o << r.num; }
};
```

- Using const correctness and const& parameters avoids copies:

```
template <class T1, class T2>
void add(const T1& a, const T2& b) { std::cout << '\n' << (a + b) << '\n'; }
```

8) Overloading templates and non-templates

- You can have:
 - A **template** add(T, T)
 - A **non-template** add(int, int) (or another overload)
- **Overload resolution rule:** when both match, a **non-template exact match is preferred** over a template (even a specialization).

```
template <class T> void add(T a, T b) { /* generic */ }
void add(int a, int b) { /* non-template overload */ }

add(4, 5); // calls non-template overload
```

- If you **force** the template with explicit arguments, it picks the specialized template path:

```
template <typename T> void print(T v) { /* general */ }
template <> void print<int>(int v) { /* specialized for int */ }
void print(int v) { /* non-template overload */ }

print(5); // calls non-template overload (preferred)
print<int>(5); // forces template specialization for int
```

9) Member function templates in a non-template class

- A non-template class can contain **template member functions**:

```
class MyClass {
public:
    void disp1() { std::cout << "\nin disp1\n"; }
    template <class T>
    void disp2(T a) { std::cout << "\n in template fun\t" << a << '\n'; }
};
```

- Works with any type you pass to disp2.

10) Class templates (generic classes)

- Basic form:

```
template <class T>
class MyClass {
    T a;
public:
    explicit MyClass(T a) : a(a) {}
    void disp() { std::cout << '\n' << a << '\n'; }
};

MyClass<int>    s1(10);
MyClass<char>   s2('A');
MyClass<double> s3(10.5);
```

- Multiple template parameters and **template inheritance** also work:

```
template <class T>
class Base {
public:
    void disp1(T a) { std::cout << a << '\n'; }
};

template <class T, class U>
class Sub : public Base<T> {
public:
    void disp2(U b) { std::cout << "in disp2" << b << '\n'; }
};

Sub<int, char> s; s.disp1(20); s.disp2('A');
```

11) Class template specialization

- You **cannot overload** a class template with a non-template class of the same name (that's a redefinition error).
- Instead, use **full specialization** for a particular type:

```
template <typename T>
class MyContainer { public: void show() { std::cout << "Generic container\n"; } };

class MyContainer<int> { public: void show() { std::cout << "Int container specialization\n"; } };
MyContainer<double> m1; m1.show(); // Generic
MyContainer<int> m2; m2.show(); // Specialized
```

- Attempting to “overload” the class name with a non-template class MyContainer{ ... }; **does not compile** (redefinition).

12) Function template overloading by arity

- You can overload a function template by **number** of parameters:

```
template <class T1, class T2>
void add(T1 a, T2 b) { std::cout << '\n' << (a + b) << '\n'; }

template <class T1, class T2, class T3>
void add(T1 a, T2 b, T3 c) { std::cout << '\n' << (a + b + c) << '\n'; }

add(2, 'A'); // two-arg version
add(2, 'A', 3.5); // three-arg version
```

13) Generic sorting examples (arrays)

- Any comparison-based sort works if the type supports the operators used (e.g., operator>):

```
template <class T>
void sort(T* p, int len) {
    for (int i = 0; i < len - 1; ++i)
        for (int j = i + 1; j < len; ++j)
            if (p[i] > p[j]) std::swap(p[i], p[j]);
}
```

- Use it for int, float, char, etc.
- (Your sample loops printed arr but declared arr1/arr2/arr3; make sure to print the actual arrays you sort.)

14) Smart-pointer-like template example (SP<T>)

- A minimal ownership wrapper:

```
template <class T>
class SP {
    T* pData;
public:
    explicit SP(T* pValue) { pData = new T(*pValue); delete pValue; }
    ~SP() { delete pData; }
    T& operator*() { return *pData; }
    T* operator->() { return pData; }
};
```

- Usage with different types:
- `SP<Person> sp1(new Person("abc", 10));`
`sp1->display(); (*sp1).shout();`

`SP<Vehicle> sp2(new Vehicle("Car", 100));`
`sp2->display(); (*sp2).start();`
- **Notes:**
 - Demonstrates **class templates** and **operator overloading**.
 - This example **takes ownership** and deep-copies. (In production, prefer `std::unique_ptr`, correct rule-of-5, etc.)

15) Template specialization vs. overloading (function case)

- **Both** can tailor behavior for certain types.
- Key rules demonstrated:
 - A **non-template overload** beats a template (even a full specialization) when it's an exact match (e.g., `print(5)` calls the non-template `print(int)`).
 - You can **force** the template's specialization with explicit template arguments: `print<int>(5)`.
- Practical takeaway:
 - If you just need one special case for int, **overloading** is simple and fine.
 - **Specialization** shines where overloading isn't available (e.g., **class templates**).

16) Additional notes and best practices from examples

- Many examples used `void main()`—**standard C++ requires** `int main()`.
- Using `const` where appropriate (e.g., `const&` parameters, `const` member operators) avoids unnecessary copies and matches `const`-correctness.
- For **mixed types** in arithmetic templates (`T1 + T2`), the result type follows C++ usual arithmetic conversions; you can make the return type explicit if needed.
- When streaming custom types, always define `operator<<`.
- The assembly/listing you showed (for `MyClass<int/char/double>`) evidences that **separate instantiations** are generated per concrete type.

Quick reference: common patterns

Function template (single type)

```
template <class T>
T add(T a, T b) { return a + b; }
```

Function template (two types)

```
template <class T1, class T2>
auto add(T1 a, T2 b) -> decltype(a + b) { return a + b; }
```

Class template

```
template <class T>
class Box {
    T value;
public:
    explicit Box(T v) : value(v) {}
    const T& get() const { return value; }
};
```

Full specialization (class)

```
template <> class Box<int> {
    int value;
public:
    explicit Box(int v) : value(v) {}
    void shout() const { std::cout << "Int box: " << value << '\n'; }
};
```

Overload beats specialization (function)

```
template <typename T> void print(T v) { std::cout << "General\n"; }
template <> void print<int>(int v) { std::cout << "Specialized int\n"; }
void print(int v) { std::cout << "Overloaded int\n"; }

print(5);           // Overloaded
print<int>(5);       // Specialized template (forced)
```

Summary

- Templates let you write **one** function/class that the compiler **instantiates** for the types you actually use.
- They **don't necessarily reduce binary size**, but they **reduce source duplication** and **centralize maintenance**.
- Use **function templates** for generic operations, **class templates** for generic containers/owners, and **specialization** when certain types require **custom behavior**—especially for **class templates** where overloading is not an option.
- Overload resolution prefers **non-template exact matches**; use explicit template arguments to force template selection.
- Ensure operands provide required operators (+, >, <=, etc.) for your instantiations to compile.

Templates and Linker Errors in C++

These notes explain why **function templates** and **class templates** cannot be distributed in compiled object libraries, and how the compiler and linker handle them.

1) Nature of Templates

- A **template** in C++ is a **blueprint** for generating functions or classes.
 - Templates are **not compiled into object code** until they are **instantiated** with specific data types.
 - When a function or class template is used with a particular type (for example, int), the compiler **generates** a concrete version of that function or class at compile time.
-

2) Compilation and Linking Process

Compilation Stage

- The compiler checks syntax and verifies whether function or class declarations exist.
- When it encounters a template definition **and** a usage (like `add(10, 20)`), it:
 - Deduces the type (int in this example).
 - Instantiates the template as `add<int>(int, int)`.
 - Generates the corresponding object code.

Linking Stage

- The linker connects all compiled object files (.obj or .o).
 - If any instantiated version of a template is missing, the linker will report an **unresolved external symbol** error.
-

3) Reason for Linker Errors with Templates

Templates cannot be compiled into object files independently because:

1. The compiler cannot know in advance **which data types** will be used in the client program.
 2. Only **template definitions**, not declarations, allow the compiler to generate instantiations.
 3. If the definition is placed in a separate source file (.cpp), it is **not visible** when another file tries to use the template.
 4. Therefore, the compiler in the client file only sees the **declaration**, assumes the instantiation exists, and defers resolution to the linker.
 5. During linking, the linker **cannot find** the matching instantiation, resulting in an error.
-

4) Why Templates Should Not Be Split Across Source Files

When you separate template declarations and definitions:

- The **header file (.h)** contains only the declaration (e.g., `template<class T> void add(T, T);`).

- The **source file (.cpp)** contains the definition.
- The **client file** includes the header and calls the template function.

Although each file compiles successfully, linking fails because:

- The compiler in the client file cannot generate instantiations without the definition.
- The object file containing the template definition only holds the **generic form**, not the instantiated code.

Thus, the linker cannot match the call `add<int>(int, int)` with any compiled version of `add`.

5) Templates in Libraries

Templates cannot be distributed as precompiled object libraries (.lib, .obj, or .o) for the following reasons:

- The **template definition** alone cannot produce object code unless instantiated.
- The **compiler** of the library does not know what types the user will use.
- Therefore, the **expanded (instantiated)** versions of templates are **not present** in the library.
- When the client links against such a library, the linker fails to find matching definitions.

Result: **Linker errors** occur because no type-specific instantiations exist inside the library.

6) Function Templates and Linker Behavior

- The compiler “mangles” function names to encode their type information.
For example:
 - A general template function might internally have a symbol like `add_type_type(type, type)`.
 - When instantiated for `int`, the compiler expects a symbol like `add_int_int(int, int)`.
 - If the function is defined in a separate library file, the linker searches for the specific mangled name but finds only the generic template symbol.
 - This mismatch leads to a **linker error**.
-

7) Class Templates and Linker Behavior

Class templates follow the same rules:

- Declaring a class template in a header without providing the definition of its member functions prevents instantiation.
- If the definitions are in a separate source file, the compiler of the client code cannot see them.
- During linking, the linker cannot find the corresponding instantiated class definitions.

Hence, **class templates** also cannot be distributed as precompiled object libraries without explicit instantiation.

8) Correct Ways to Distribute Template Code

(A) Define Templates in Header Files

- The most common and recommended method.
- Place both the **declaration** and **definition** in the same header file.
- Include this header wherever the template is used.

Advantage:

- The compiler always has access to the full definition and can instantiate the required versions as needed.

Example conceptually:

```
// myheader.h
template<class T>
void add(T a, T b) {
    cout << a + b << endl;
}
```

(B) Use Explicit Instantiation

- If it is necessary to keep the definition in a separate source file, use **explicit instantiation** statements.

Conceptually:

```
// add.cpp
template void add<int>(int, int);
template void add<double>(double, double);
```

This tells the compiler to generate object code for the specified types so that the linker can find them later.

10) Practical Guidelines

1. **Always keep template definitions in header files** unless explicit instantiation is planned.
 2. **Avoid separating** declarations and definitions of templates across different source files without proper instantiation.
 3. **Do not distribute** template code as compiled object libraries unless all possible type instantiations are explicitly included.
 4. **Understand that templates are compile-time constructs** — the compiler must see the complete definition to generate usable code.
 5. **Linker errors** in templates typically indicate that the compiler could not find or generate the required instantiation.
-